Algorithmes, Types, Preuves: Présentation du projet: Python vers JavaScript

Martin Strecker

Univ. J.-F. Champollion

Année 2025/2026

Plan

1 Présentation Projet

2 Git

Plan

1 Présentation Projet
- Analyse et traduction d'un langage impératif
- Python
- Python annoté avec des types
- Langage source
- Typage
- Génération de JavaScript
- Analyse syntaxique
- Prise en main

Survol du projet

Le développement à faire :

- Formalisation de la syntaxe du langage Python en Caml ($\rightsquigarrow$ Langages et automates)
- Vérification de types de ce langage
- Traduction vers JavaScript

Survol du projet

Motivation et but d'apprentissage :

- Savoir transférer les concepts vus pour un langage fonctionnel vers un langage impératif (surtout : typage)
- Faire connaissance de la représentation interne de ce langage.
- S'initier aux arcanes de Python.
- S'initier à des concepts de compilation

Plan

1 Présentation Projet
- Analyse et traduction d'un langage impératif
- Python
- Python annoté avec des types
- Langage source
- Typage
- Génération de JavaScript
- Analyse syntaxique
- Prise en main

Python

Langage prétendument simple :

- au niveau syntaxique :
  - syntaxe épurée, peu de mots-clés
  - blocs de code marqués par indentation
    au lieu de “parenthésage” avec `begin .. end` ou `{ .. }`
- au niveau du typage :
  - pas de déclaration de variables
  - pas d’annotation de types requises
  - pas de vérification de types apparente
    $\rightsquigarrow$ pas de rejet de programmes mal typés
- au niveau de la sémantique : *anything goes*

. . . qui cache en vérité une complexité effarante :

- Comportement d’un programme souvent difficile à prédire
- Prévention d’erreurs difficile. Langage à haut risque.

Variables locales vs. globales

Quelle valeur est renvoyée par l'appel `foo(4)` ?

```
y = 3
def foo(x):
    return(x+y)
foo(4)
```

Quelle valeur est renvoyée par l'appel `foo(4)` ?

```
y = 3
def foo(x):
    y = 5
    return(x+y)
foo(4)
```

Et quelle est la valeur de `y` après l'appel ?

Opération +

```
def plus(x, y):
    return(x + y)
```

Quelles sont les valeurs renvoyées par :

- `plus(3, 4)`
- `plus("ab", "cd")`

Mêmes questions pour le programme

```
def plus(x, y):
    return(x + y + 2)
```

Opérations de comparaison

```
def qd(x,y):
    if x == y:
        return(x)
    else:
        return(y)
```

- Quel est le résultat de l'appel `qd(42, "quarantedeux")` ?
- Et si on remplace == par <= ?

Opérations booléennes

- Quelle est la valeur de `(True and 42) < 40` ?
- Quelle est la valeur de `(42 and True) < 40` ?
- Dans le programme suivant, quelle valeur est affichée pour l'appel `bar(13)` ?

```
def oubli_return(x):
    y = x

def bar(x):
    if oubli_return(x):
        print(1)
    else:
        print(2)
```

Plan

1 Présentation Projet
- Analyse et traduction d'un langage impératif
- Python
- Python annoté avec des types
- Langage source
- Typage
- Génération de JavaScript
- Analyse syntaxique
- Prise en main

Annotations de types

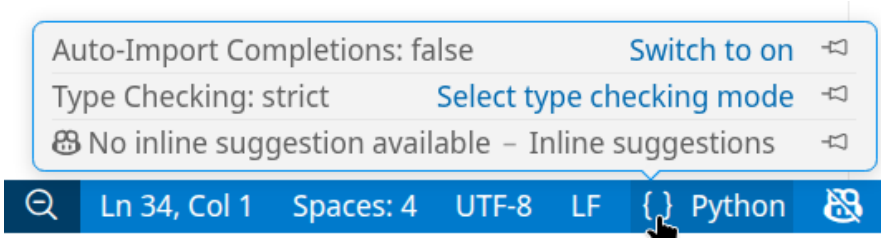
Dans ce projet,

- toute variable doit être déclarée
- les variables et fonctions doivent être annotées avec des types

```
def qd(x : int | str, y: str) -> int | str:
    n : str
    n = "nombre␣"
    if x == y:
        return(n + x)
    else:
        return(n + y)
```

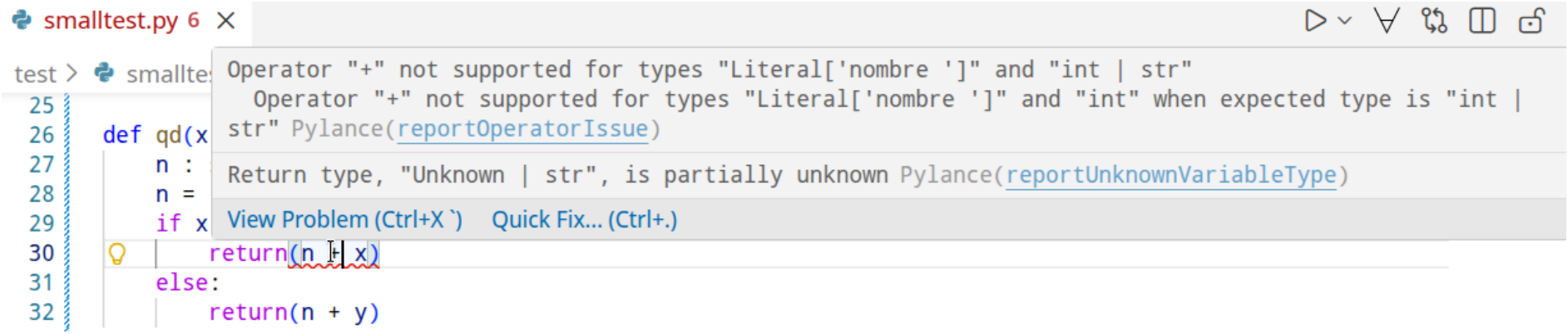
Vérification de types avec VSCode

Dans VSCode, on peut sélectionner un mode de vérification strict :



(alternative : `Ctrl+,` (control et virgule), chercher `typechecking`)

On aura alors des messages d'erreur détaillés :



Système de typage de Python : Graduel

Voir aussi la spécification (peu formelle) :
`https://typing.python.org/en/latest/spec/index.html`
Le typage en Python est *très complexe* :

- a priori : typage *dynamique*
- les annotations de type permettent une vérification partielle
  les types sont introduits graduellement.
- mais une variable peut avoir un type indéterminé et changer
  $\rightsquigarrow$ typage *graduel*

```
x = "abc" # x a le type str
x = 3     # x a le type int, incompatible avec str
```

Dans notre modélisation :

- Toute variable doit être déclarée explicitement.
- Pas de typage graduel.
- Pas de traitement de classes / traits orientés objet

Système de typage de Python : Union

Python admet des types union : `T1 | T2 | ...`
Une variable d'un type union peut "porter" toute valeur d'un type constituant :

```
x : str | int
x = "abc" # OK, str inclus dans str | int
x = 3     # OK, int inclus dans str | int
x = 1.5   # incorrct, float pas inclus dans str | int
```

Nous modélisons les types union

Système de typage de Python : Compatibilité

Un type peut être compatible avec un type différent qui représente un domaine plus large :

```
x : float
x = 3      # OK, int compatible avec float
x = True   # OK, bool compatible avec float et int
```

La compatibilité n'est pas symétrique

```
b: bool
b = True   # OK
b = 2      # incorrect, int pas compatible avec bool
```

Nous modélisons la compatibilité

Système de typage de Python : Surcharge

Deux formes de surcharge :

1. opérateur manipule des types non compatibles, sans rapport sémantique *Exemple :*
   - Addition numérique : `3 + 4 = 7`
   - Concaténation de `str` : `"abc" + "cd" = "abccd"`
2. opérateur manipule des types compatibles :
   - Addition entre `int` et `float` convertie automatiquement : `3.2 + 4 = 7.2`
   - Addition entre `int` et `bool` avec `True` interprété comme 1 : `3 + True = 4;  5 * False = 0`

Nous modélisons la surcharge

Système de typage de Python : Type `None`

`None` indique la terminaison d'une fonction / procédure sans résultat :

```
def foo(x: int) -> int | None:
    if x > 0:
        return(x)
```

Cette analyse permet de détecter des définitions partielles.
Nous utilisons le type `None`

```
def abs(x: int) -> int:
    if x > 0:
        return(x)
        x = 0 - x  # inatteignable. Pb. d'indentation?
    return(x)
```

Détection de code inatteignable lors de la vérification de types

Système de typage de Python : Autres analyses

D'autres analyses assimilables à des types sont effectués :

`Unbound` pour des variables non initialisées

```
def bar1(x: int) -> int:
    y : int
    return(x+y)          # y Unbound
```

`not accessed` si la valeur n'est pas utilisée :

```
def bar2(x: int) -> int:
    y : int
    y = 2                # y not accessed
    return(x)
```

Ces analyses ne sont pas traitées ici

Plan

1 Présentation Projet
- Analyse et traduction d'un langage impératif
- Python
- Python annoté avec des types
- Langage source
- Typage
- Génération de JavaScript
- Analyse syntaxique
- Prise en main

Syntaxe : Expression

Distinction entre *expression* (calcule une valeur) et *statement* (manipule un état : affectation de valeurs à des variables)

```
type expr =
    Const of value                          (* constant *)
  | VarE of vname                           (* variable *)
  | BinOp of  binop * expr * expr           (* binary op *)
  | CallE of vname * (expr list)            (* function call *)
```

Exemple : représentation de $f(n-1)$:

```
CallE ("f",
       [(BinOp ((BArith BAsub),
                (VarE "n"),
                (Const (IntV 1))))
       ])
```

Syntaxe : Statement

```
type stmt =
    Block of stmt list
  | Assign of vname * expr        (* Assignment: x = e *
  | Cond of expr * stmt * stmt    (* if .. then .. else
  | While of expr * stmt
  | Return of  expr
  | CallS of vname * (expr list)  (* procedure call *)
```

Différence entre

- `CallE` : appel de fonction, comme dans `x = f(y)`
- `CallS` : appel de procedure, comme dans `p(x,y)`

Syntaxe : Statement

Exemple : représentation de :

```
if (x < 0):
   return(0 - x)     # pas d'inverse unaire
return(x)
```

En Caml :

```
(Block
   [(Cond ((BinOp ((BCompar BClt), (VarE "x"),
                  (Const (IntV 0)))),
           (Block
                  [(Return
                     (BinOp ((BArith BAsub),
                      (Const (IntV 0)), (VarE "x"))))]),
           (Block [])));
    (Return (VarE "x"))])
```

Syntaxe : Fonctions

```
(* function declaration: function name;
    parameter declarations; return type *)
type fundecl = Fundecl of fname * (vardecl list) * tp

(* function definition: function declaration;
   local var decls; function body *)
type fundefn =
   Fundefn of fundecl * (vardecl list) * stmt
```

Représentation de :

```
def abs(x: int) -> int:
    if (x < 0): ...
```

```
Fundefn ((Fundecl ("abs",
                   [(Vardecl ("x", (UnionT [IntT])))],
                   (UnionT [IntT]))),   [],
         (Block [(Cond (....))]))
```

Syntaxe : Programme

```
(* program: function definitions;
         global var decls; statement *)
type prog =
   Prog of fundefn list * (vardecl list) * stmt
```

Plan

1 Présentation Projet
- Analyse et traduction d'un langage impératif
- Python
- Python annoté avec des types
- Langage source
- Typage
- Génération de JavaScript
- Analyse syntaxique
- Prise en main

Environnements

L'environnement de typage est plus complexe que pour Mini-Caml

```
type var_environment = {
    globals: (vname * tp) list;
    locals: (vname * tp) list;
    }
type environment = {
    fdecls: (vname * ((tp list) * tp)) list;
    static_vars: var_environment;
    dyn_vars: var_environment;
    curfun: fname option;
    }
```

Déclarations de fonction

```
fdecls: (vname * ((tp list) * tp)) list;
```

Pour toute déclaration de fonction, on retient :

- son nom
- les types de ses paramètres
- son type de retour

Exemple : pour la fonction

```
def abspart(x: int) -> int | None:  ...
```

on obtient :

```
("abspart",
  ([(UnionT [IntT])], (UnionT [IntT; NoneT])))
```

Fonction actuelle

```
curfun: fname option;
```

Mémoriser si on est à l'intérieur d'une fonction, et si oui, laquelle :

- `None` si on fait la vérification des instructions globales
- `Some "f"` si on vérifie le corps de la fonction `f`

Dans chaque cas, le comportement est différent.

Variables locales vs globales

```
def foo() -> int | str:
    # x : str
    # x = "trois"
    return(x)
x : int
x = 3
```

- Le `x` de `return(x)` se réfère à la variable globale.
  Le résultat de l'appel `foo()` est 3
- Si on décommente les lignes 2 et 3 : `x` se réfère à la variable locale.
  Le résultat de l'appel `foo()` est `"trois"`.

Obtenir le type d'une variable

```
type var_environment = {
    globals: (vname * tp) list;
    locals: (vname * tp) list;
  }
```

Pour obtenir le type d'une variable :

- On cherche d'abord dans `locals`
- Si variably n'y est pas : on cherche dans `globals`

Environnements dynamiques / statiques

Le type d'une variable varie selon son utilisation :

- En mode écriture (affectation) : On peut affecter toute valeur compatible avec le type déclaré (statique) de la variable

```
x : int | str
x = 3           # legal
x = "trois"     # legal
x = 3.4         # illegal
```

Environnements dynamiques / statiques

- En mode lecture (dans une expression)

```
x : int | str
x = 3
y = x + 1       # legal
y = x + "un"    # illegal, x a le type int ici
x = "trois"
y = x + "un"    # legal, x a le type str ici
```

  Le type dynamique d'une variable change en fonction des affectations précédentes

Pour cela, deux environnnements de variables :

```
static_vars: var_environment;
dyn_vars: var_environment;
```

Faire la vérification d'un programme

Fonction `tp_prog : prog -> bool`

1. Récupérer les en-têtes des fonctions et construire `fdecls` (qui ne changera plus)
2. Construire l'environnement de variables globales. Initialement, `static_vars` = `dyn_vars`. Vérifier les instructions globales.
3. Vérifier chacune des fonctions.

Le résultat de `tp_prog` est sans véritable intérêt :

- Erreur de typage (quelque part) : on lève une exception
- Programme bien typé : la fonction termine normalement

Faire la vérification d'une fonction

Fonction `tp_fundefn : environment -> fundefn -> bool`

1. On rajoute les types des variables locales à l'environnement. Initialement, `static_vars` = `dyn_vars`.
   (variables locales = paramètres de la fonction et déclarations de variables locales)
2. On vérifie qu'il n'y a pas deux variables locales avec le même nom.
3. On vérifie le corps de la fonction (un `statement`)
   ... dont le type doit être compatible avec le type résultat de la fonction

Faire la vérification d'une expression

Fonction `tp_expr : environment -> expr -> tp`

1. On vérifie les sous-expressions récursivement
2. ... et assure que les opérateurs / fonctions sont appliqués avec le bon type pour toute combinaison de types

Exemples :

- `x: int, y: int` Correct : `x + y`
- `x: str, y: str` Correct : `x + y` ; incorrect : `x - y`
- `x: int | str, y: int | str`
  Incorrect : `x + y`
  (parce que mal typé pour la combinaison `x: int`, `y: str`)
  Correct : `x == y` ; incorrect : `x < y`
  (== s'applique à toute combinaison de type, mais pas <).

Faire la vérification d'un statement

Fonction `tp_stmt` qui doit :

- mettre à jour l'environnement dynamique :

```
# Environnement dynamique: { x: int | str, y : int}
x = 3
# Environnement dynamique: { x: int, y : int}
x = 'trois'
# Environnement dynamique: { x: str, y : int}
```

- Calculer un type résultat :

```
x = 42
# si la fonction s'arrete ici, le resultat est: None
return(x)
# si la fonction s'arrete ici, le resultat est: int
```

- Et que faire avec du code après un `return` ?

```
return(x)
x = 43    # instruction inatteignable
```

Faire la vérification d'un statement

Fonction `tp_stmt : (environment * tp * bool) -> stmt -> (environment * tp * bool)`
fait glisser un triplet :

- environnement actuel
- type résultat
- booléen "après `return`"

sur les instructions

Séquence de statement

Séquence linéaire d'instructions (`Block`). Uniquement la partie dynamique de l'env. est montrée :

```
# ({ x : int | str, y : float}, None, false)
x = 42
# ({ x : int, y : float}, None, false)
y = 13          # 13 : int est compatible avec float
# ({ x : int , y : int}, None, false)
return(x)
# ({ x : int , y : int}, int, true)
x = 43
# levee d'exception, code inatteignable
```

if .. else

Branchement : fusionner les environnements des deux branches

```
# ({ b : bool, x : int | str, y : float}, None, false)
if b:
    # ({ b : bool, x : int | str, y : float}, None, false)
    x = 3
    y = 1
    # ({ b : bool, x : int, y : int}, None, false)
else:
    # ({ b : bool, x : int | str, y : float}, None, false)
    x = 'trois'
    y = 1.5
    # ({ b : bool, x : str, y : float}, None, false)
# ({ b : bool, x : int | str, y : float}, None, false)
return(x + y)
# x + y mal type -> exception
```

while

Est-ce que `x3` est toujours de type `str` à la fin de la boucle ?
1er parcours boucle :

```
def f (i: int, x1: int | str, n: int, s: str) -> str:
    x2 : int | str
    x3 : int | str
    x1 = s;  x2 = s;  x3 = s
    while (i > 0):
        # {x1: str, x2: str, x3: str}
        x3 = x2
        # {x1: str, x2: str, x3: str}
        x2 = x1
        # {x1: str, x2: str, x3: str}
        x1 = n
        # {x1: int | str, x2: str, x3: str}
        i = i - 1
    return(x3)
```

while

Est-ce que `x3` est toujours de type `str` à la fin de la boucle ?
2ème parcours boucle :

```
def f (i: int, x1: int | str, n: int, s: str) -> str:
    x2 : int | str
    x3 : int | str
    x1 = s;  x2 = s;  x3 = s
    while (i > 0):
        # {x1: int | str, x2: str, x3: str}
        x3 = x2
        # {x1: int | str, x2: str, x3: str}
        x2 = x1
        # {x1: int | str, x2: int | str, x3: str}
        x1 = n
        # {x1: int | str, x2: int | str, x3: str}
        i = i - 1
    return(x3)
```

while

Est-ce que `x3` est toujours de type `str` à la fin de la boucle ?
3ème parcours boucle :

```
def f (i: int, x1: int | str, n: int, s: str) -> str:
    x2 : int | str
    x3 : int | str
    x1 = s;  x2 = s;  x3 = s
    while (i > 0):
        # {x1: int | str, x2: int | str, x3: str}
        x3 = x2
        # {x1: int | str, x2: int | str, x3: int | str}
        x2 = x1
        # {x1: int | str, x2: int | str, x3: int | str}
        x1 = n
        # {x1: int | str, x2: int | str, x3: int | str}
        i = i - 1
    return(x3)
```

while

Est-ce que `x3` est toujours de type `str` à la fin de la boucle ?
4ème parcours boucle (point fixe) :

```
def f (i: int, x1: int | str, n: int, s: str) -> str:
    x2 : int | str
    x3 : int | str
    x1 = s;  x2 = s;  x3 = s
    while (i > 0):
        # {x1: int | str, x2: int | str, x3: int | str}
        x3 = x2
        # {x1: int | str, x2: int | str, x3: int | str}
        x2 = x1
        # {x1: int | str, x2: int | str, x3: int | str}
        x1 = n
        # {x1: int | str, x2: int | str, x3: int | str}
        i = i - 1
    return(x3)
```

Réponse : `x3` *peut* aussi avoir le type `int`, donc `f` est mal typée

while

Rappel :

- La vérification de types doit détecter des problèmes pour *toute* exécution possible,
- sans connaissance précise des valeurs (nombre `i` d'itérations ...)

Conclusion de la discussion : Pour calculer l'environnement après une boucle :

- démarrer avec l'environnement initial
- exécuter corps de la boucle (symboliquement)
- prendre l'union de l'environnement avant et après exécution
- jusqu'à stabilisation (point fixe)

Pourquoi est-ce qu'on atteint un point fixe ?
Croissance monotone des types de l'environnnement, qui sont bornés.

Plan

1 Présentation Projet
- Analyse et traduction d'un langage impératif
- Python
- Python annoté avec des types
- Langage source
- Typage
- Génération de JavaScript
- Analyse syntaxique
- Prise en main

Idée

A partir du programme Python :

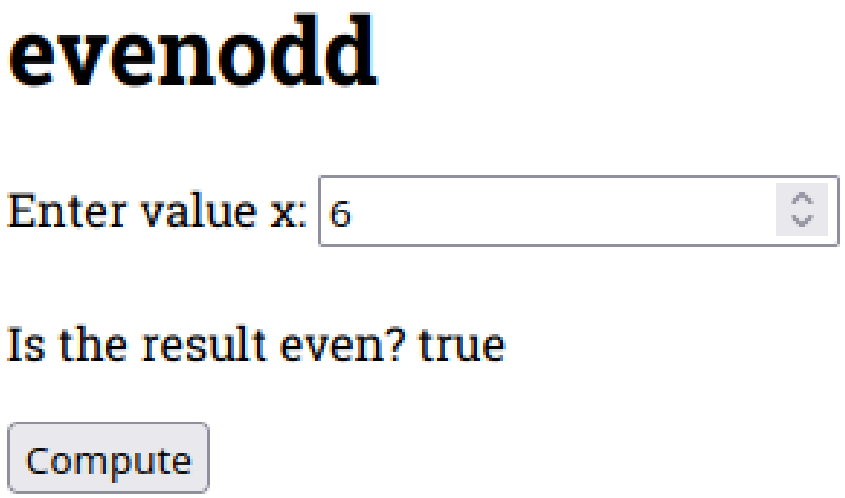
```
# Fonctions
def is_even(n: int) -> bool:
    return(n == 0 or is_odd(n - 1))

def is_odd(n: int) -> bool:
    return((n != 0) and is_even(n - 1))

# Programme principal
x: int
x = int(input("Enter value x: "))
print("Is the result even? ", is_even(x))
```

Idée

Générer du JavaScript dans une page `html` :



Détails

... laissés à votre appréciation !

A prendre en compte :

- Python et JS sont très proches mais :
  - différences de syntaxe, par ex. `or/and` (Python) et `|| / &&` (JS)
  - différents systèmes de types, par ex. `int/float` en Python, un seul type numérique en JS
  - impact sur la sémantique ? A vous d'explorer ...

Vous pouvez supposer :

- que les fonctions du programme sont "pures" (sans entrées-sorties)
- il y a un nombre limité de primitives d'interaction :
  - `input`, `print`
  - dans le programme principal
  - pour lesquels is faut développer des traducteurs `html` / JS dédiés

Implantation

Utilisation de la librairie `PPrint`.

Idée :

- création d'un document à l'aide de primitives de haut niveau :
  - `parens` *doc*, mieux que :
    `"(" ^ doc ^ ")"`
  - `separate comma` *xs*
    affiche les éléments de *xs* séparés par une virgule
  - `nest 4` *doc*
    affiche *doc* (p-ê plusieurs lignes) indenté de 4 positions
- conversion du document en `string` et affichage

Voir première proposition dans le fichier `pprinter.ml`.

Implanter `doc_of_expr`, `doc_of_stmt` etc.

Plan

1 Présentation Projet
- Analyse et traduction d'un langage impératif
- Python
- Python annoté avec des types
- Langage source
- Typage
- Génération de JavaScript
- Analyse syntaxique
- Prise en main

But de votre travail

En bref :

- Ecrire un qui reconnaît l'esserntiel du langage standard Python
- . . . mais pas tout

En détail :

- Les programmes acceptés par le parser doivent être syntaxiquement corrects
  ($\rightsquigarrow$ possibilité d'utiliser d'autres parsers / interpréteurs de Python)
- Le parser ne reconnaît pas *tout* le langage standard
  Exemples : programmation OO ; boucles `for` ; entrées - sorties
- . . . mais uniquement des programmes conformes à notre syntaxe abstraite
- Quelques restrictions s'appliquent :
  - Toutes les fonctions sont définies avant le programme principal
  - Selon le lexer : commentaires `#begin`...`#end` au lieu d'indentation
  - Uniquement des programmes typées

Prise en main du parser

Note :

- Un parser opérationnel vous est déjà fourni : `PythonToJS/lib/parser.ml`
- ... et vous devez écrire votre propre parser avec Menhir

Préparar l'utilisation de Menhir :

- Récupérer `parser.mly` de Moodle
- Le mettre dans le répertoire `PythonToJS/lib`
- Renommer `lib/parser.ml` en `lib/parser_orig.ml`
- dans `lib/dune`, décommenter les lignes `(menhir ...)`
- `dune build`

(Pour revenir au parser existant : Renommer `parser.mly` en `parser_orig.mly` et `parser_orig.ml` en `parser.ml`)

Exécution / test du parser

Deux modes d'exécution :

- lecture d'un fichier (option `f` :
  `dune exec PythonToJS f test/evenodd.py`
- lecture interactivement de la console (option `i`) :
  `dune exec PythonToJS i`
  - terminer saisi d'un programme avec `ctrl-d`
  - terminer lecture interactive avec `ctrl-c`
- Exemple : `x = 2`

A noter : Il faut recompiler le parser avec `dune build` après chaque modification de `parser.mly`

Plan

1 Présentation Projet
- Analyse et traduction d'un langage impératif
- Python
- Python annoté avec des types
- Langage source
- Typage
- Génération de JavaScript
- Analyse syntaxique
- Prise en main

Projet fourni

Un projet est fourni avec :

- Une structure complète :
  - `bin/main.ml` Programme principal
  - `lib/auxdefs.ml` Fonctions auxilaires (si besoin)
  - `lib/interf.ml` interface avec parser
  - `lib/lang.ml` structures de données du langage
  - `lib/pprinter.ml` *pretty printing*
  - `lib/typing.ml` fonctions de typage
  - `test/evenodd.py` exemple de programme Python
  - `dune-project` et `*/dune` pour la compilation
- Un code incomplet

Premiers essais

- Récuper `pythontojs.tgz`
- L'extraire : `tar xfz pythontojs.tgz`
- Si vous travaillez avec un système de gestion de version (Git etc.) :
  y mettre tous les fichiers
- Compiler : `dune build`
  Crée (entre autres) un répertoire `_build`
- Compiler : `dune exec PythonToJS f test/evenodd.py`

En général : Ne pas mettre de fichiers générés sous Git :

- Inutile
- Conflits interminables
- Taille : fichiers source : $<$ 100 KB, `_build:` $>$ 30 MB

Plan

1 Présentation Projet

2 Git

Plan

2 Git
- Git : the version control system
- Actions on a single branch
- Using several branches
- Github

Why version control ?

If you are working alone on a software project, a document, . . . :

- it can happen that you make a mistake (e.g. delete part of your code) and want to restore a previous version
- you want to try out new features which maybe won't work
- you have several product lines (deployed at customer A, B, C, and development line) requiring bug fixes

. . . so a version control system is for you

Why version control ?

If you are working with others :

- you want to keep track of who made which changes (documentation !)
- if two developers make changes
  - in different files F_1, F_2,
  - or changes in different places of the same file F,

  then both changes should survive and not be overwritten
- if two developers make changes at the same place in a file, then you have to resolve a conflict

. . . so a version control system is for you

Introducing Git

In this talk : mostly Git

- one of the most widespread VCS (*) in use
- with excellent tool support
- and a prominent server platform : *Github*

We will see that :

- Git is not the only VCS
- Git (a software system) $\neq$ Github (a platform)

. . . so please experiment with alternatives and make your choice

(*) VCS : Version Control System

Introducing Git

Documentation :

- Website : `https://git-scm.com/`
  with detailed reference : `https://git-scm.com/docs`
- Book : ProGit, by Scott Chacon and Ben Straub, 2014 (but still valid)
  `https://git-scm.com/book/en/v2`

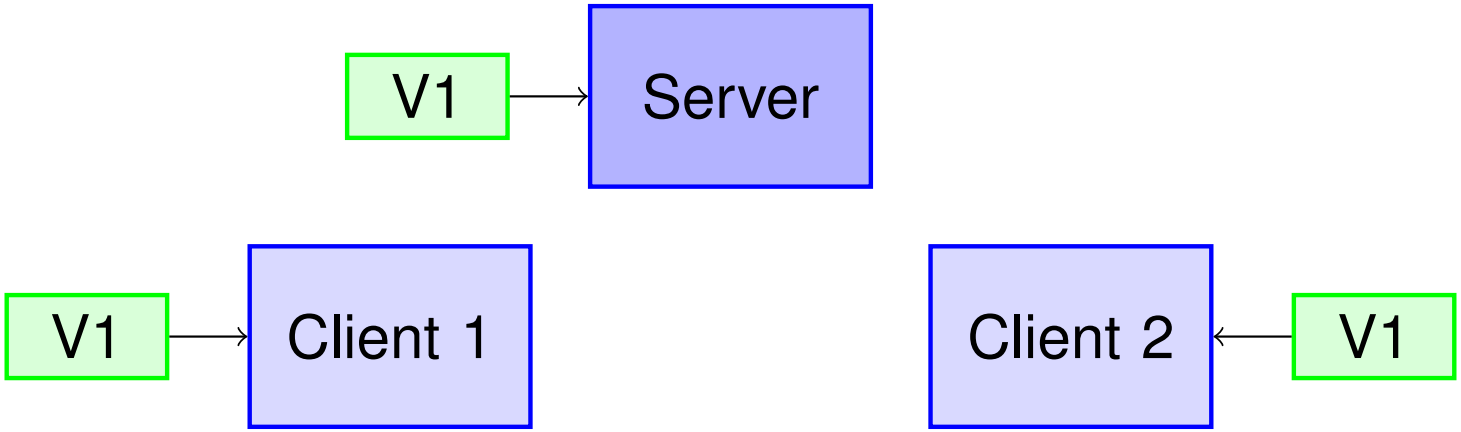
Some history : Centralized VCS

Precursors of Git are

- SVN (Subversion) released 2000
- CVS (Concurrent Versions System) released 1986
- . . . and some still older ones

Basic setup :

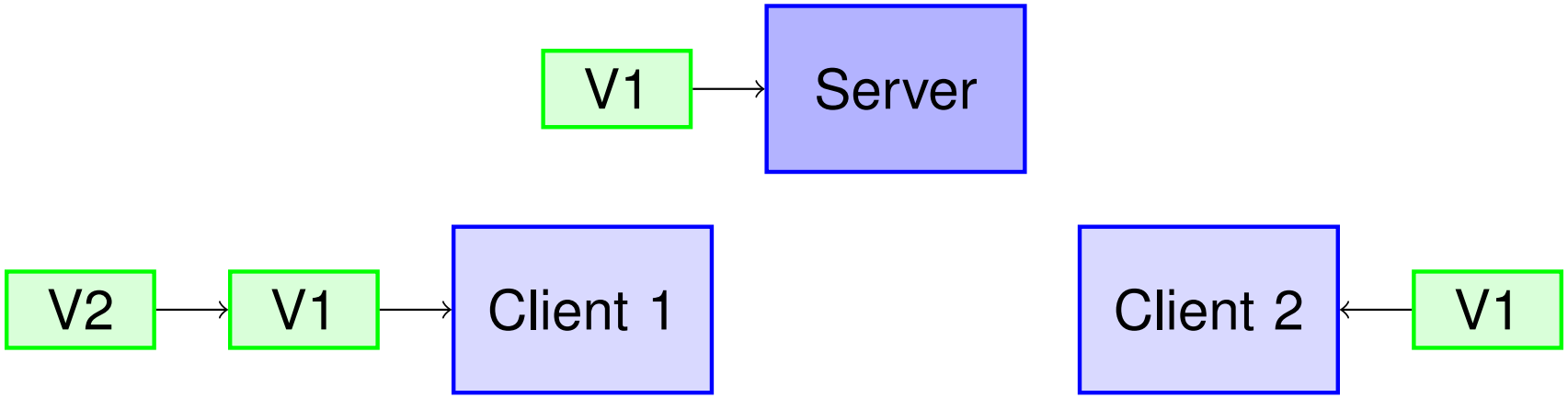
- One centralized server, several clients
- Initially all having the same version of the file



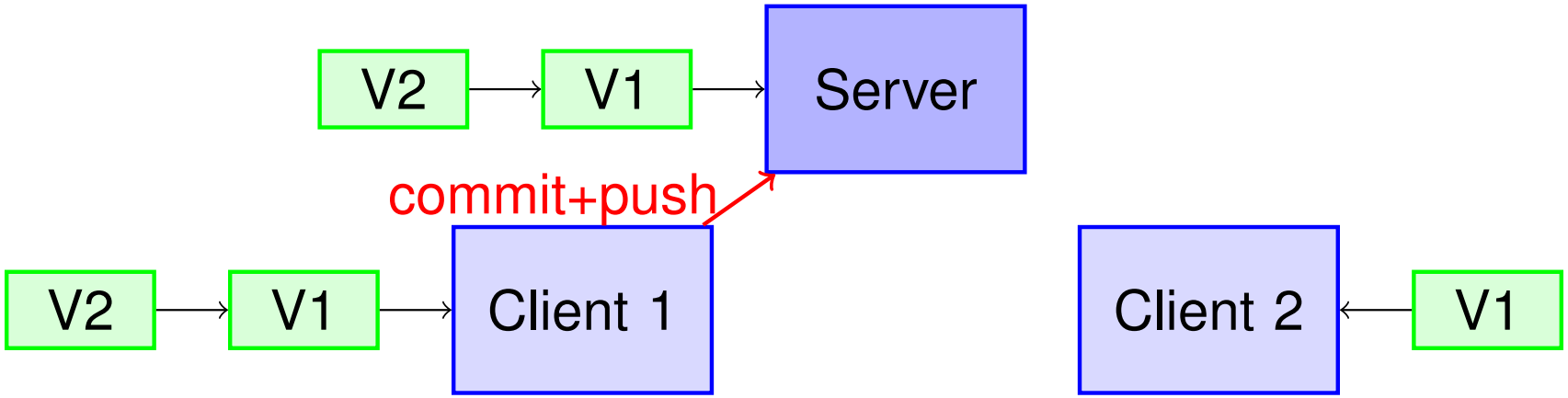
Some history : Centralized VCS

Typical development cycle :

- Client 1 develops V2 of system

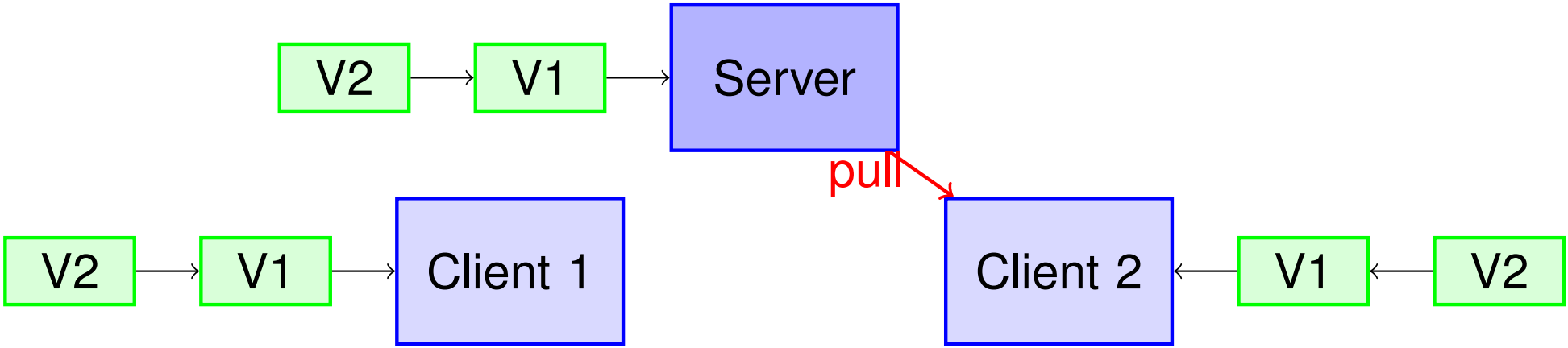


- then does a `commit+push` to the Server



Some history : Centralized VCS

- From the Server, Client 2 can `pull` this version
- (or Client 1 develops a V3 and commits+pushes it)

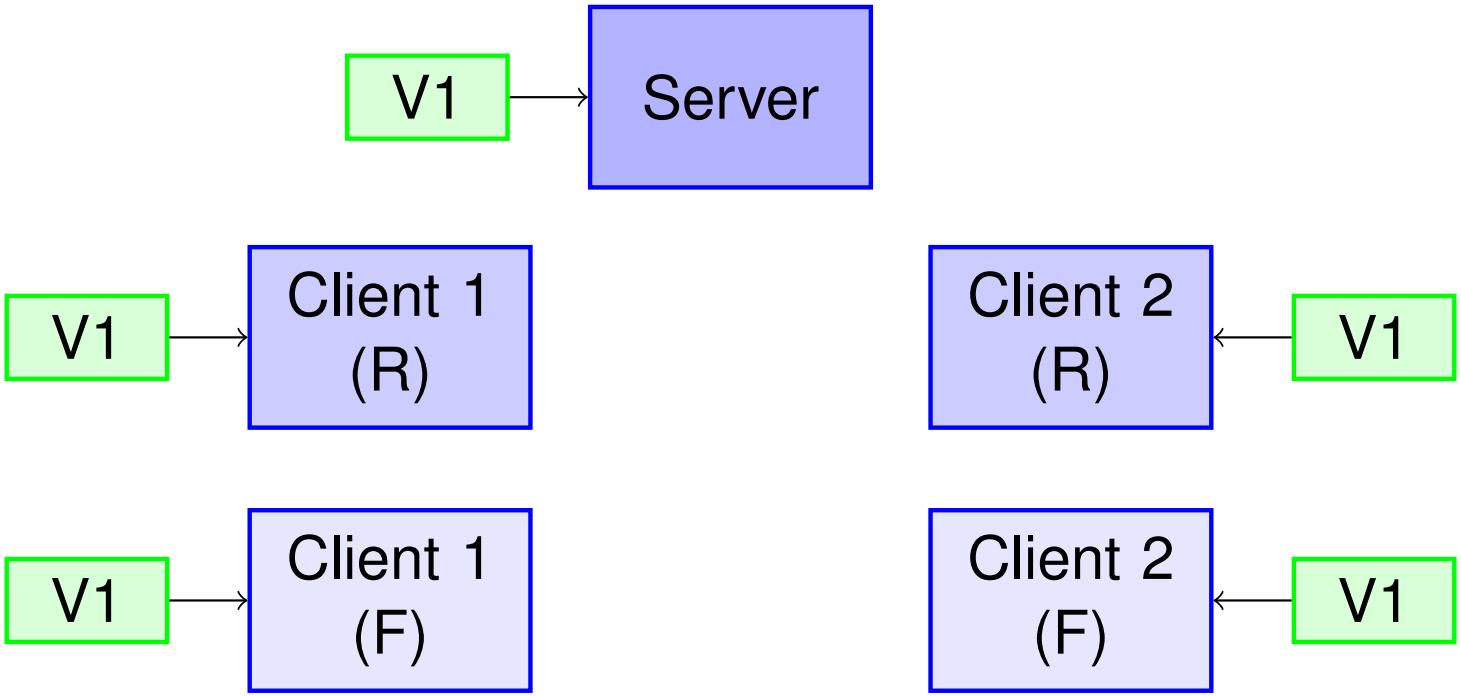


Some history : Decentralized VCS

- Darcs (released in 2003)
- Git and Mercurial (released almost on the same day in 2005)

Essential difference *wrt.* centralized system :

- Each client has its local repository (Client 1 (R), Client 2 (R))
- . . . and also the data on its file system (Client 1 (F), Client 2 (F))
- (in fact, each client can act as server)



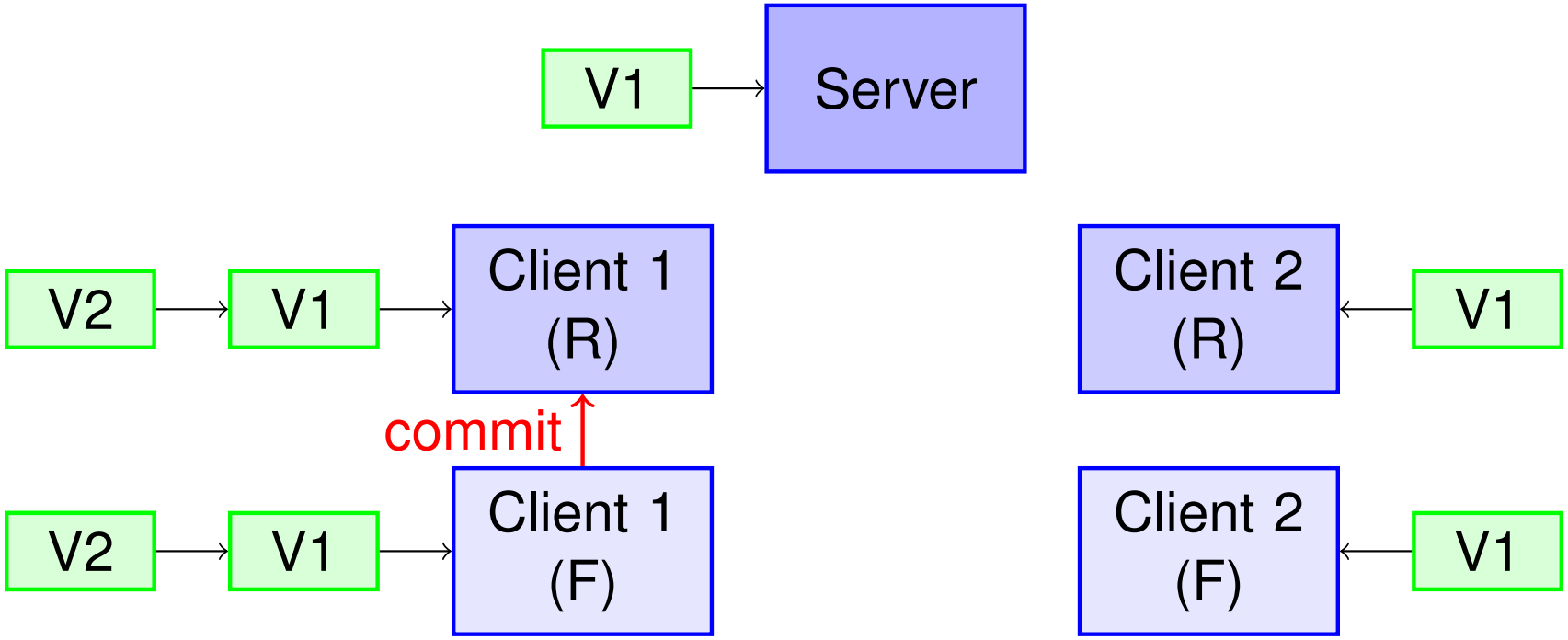
Plan

2 Git
- Git : the version control system
- Actions on a single branch
- Using several branches
- Github

Development cycle in a decentralized VCS

Typical development cycle :

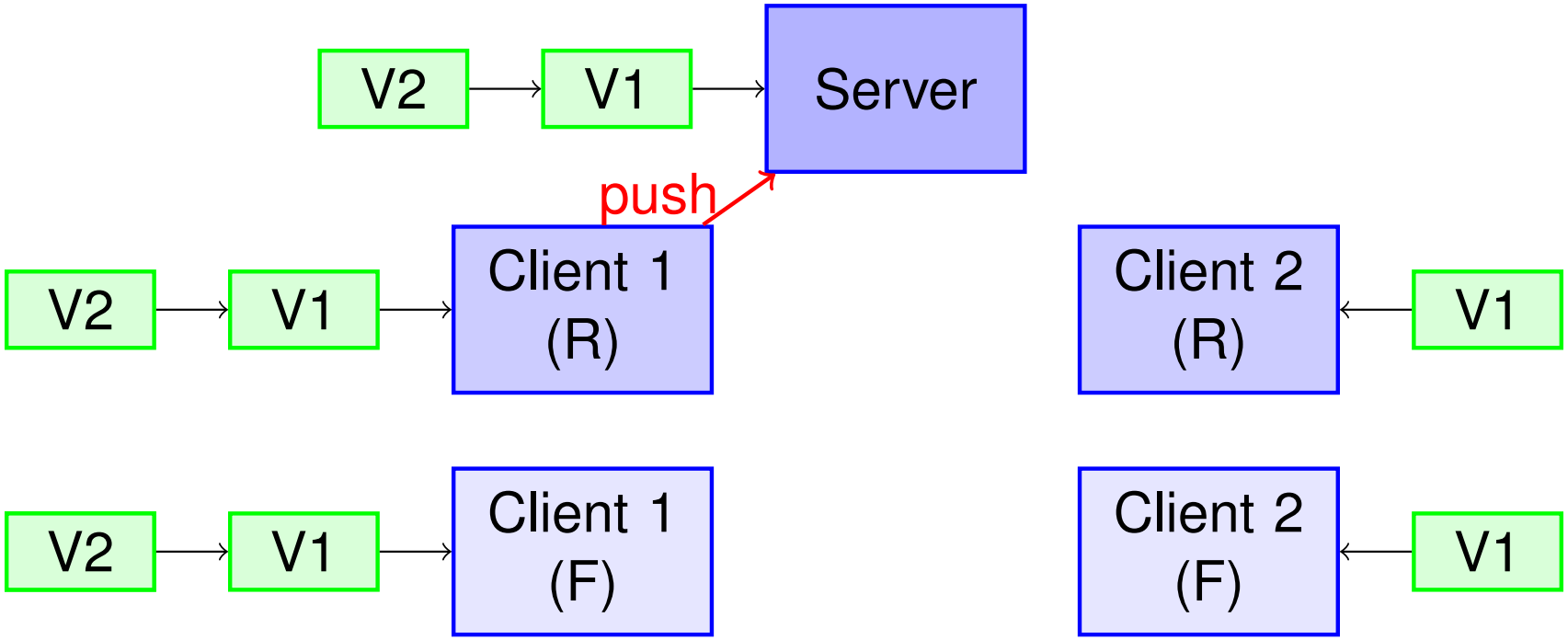
- Client 1 develops V2 of system



- then does a `commit` to the local repository
- possibly several times before transmitting to the server

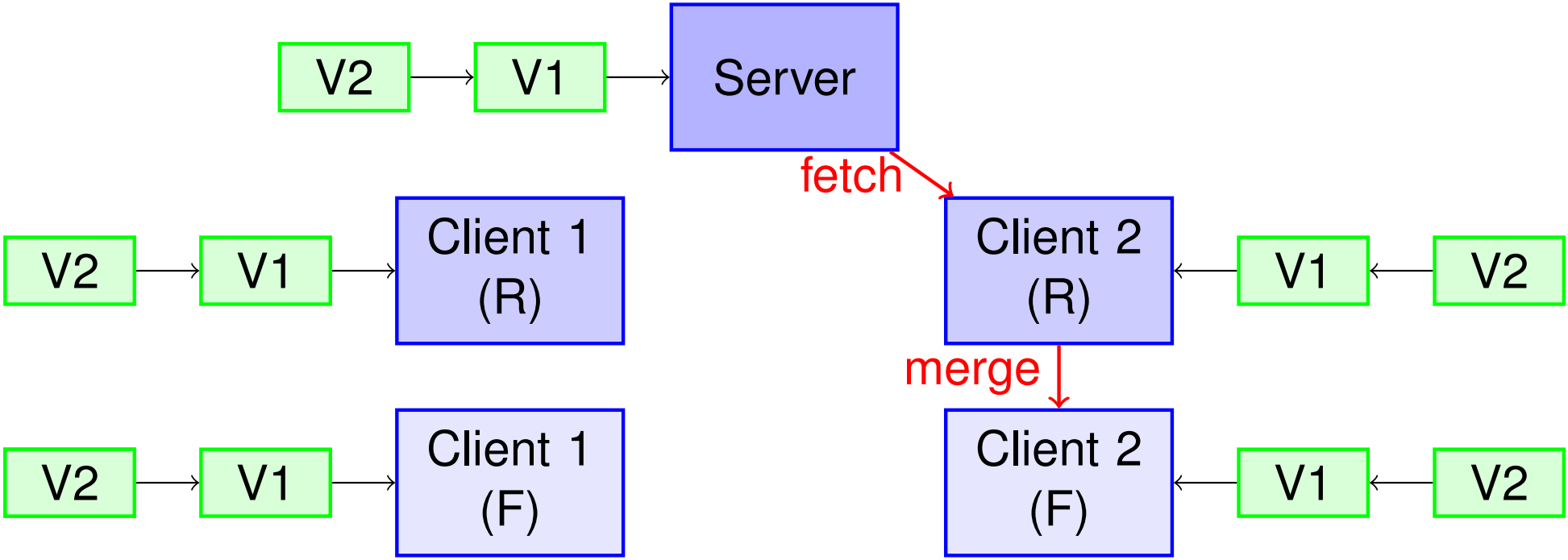
Development cycle in a decentralized VCS

- Finally, the local repository is pushed to the server.
- It is only at this moment that the changes become visible to other clients.



Development cycle in a decentralized VCS

- The new version can now be pulled from the server by the other clients.



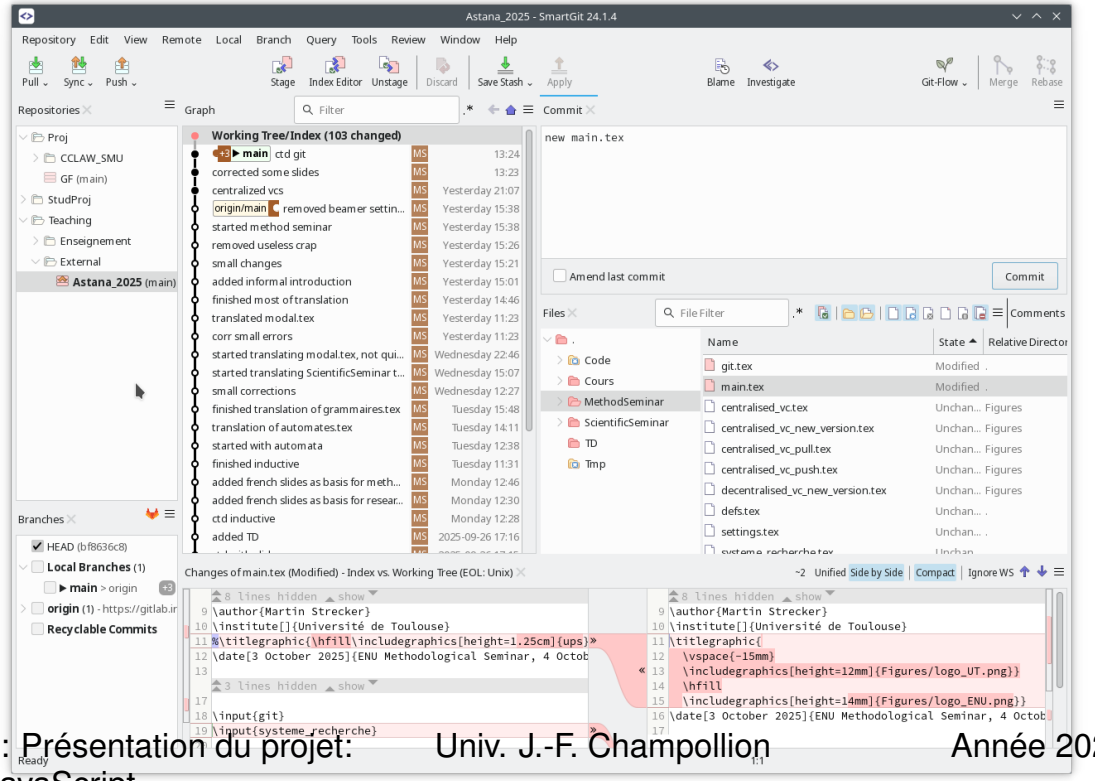
Note :

- In Git, `fetch` and `merge` are typically done in one step with a `pull`
- In Mercurial, these operations are kept separate and called `pull`

Interacting with Git

Git can be run through

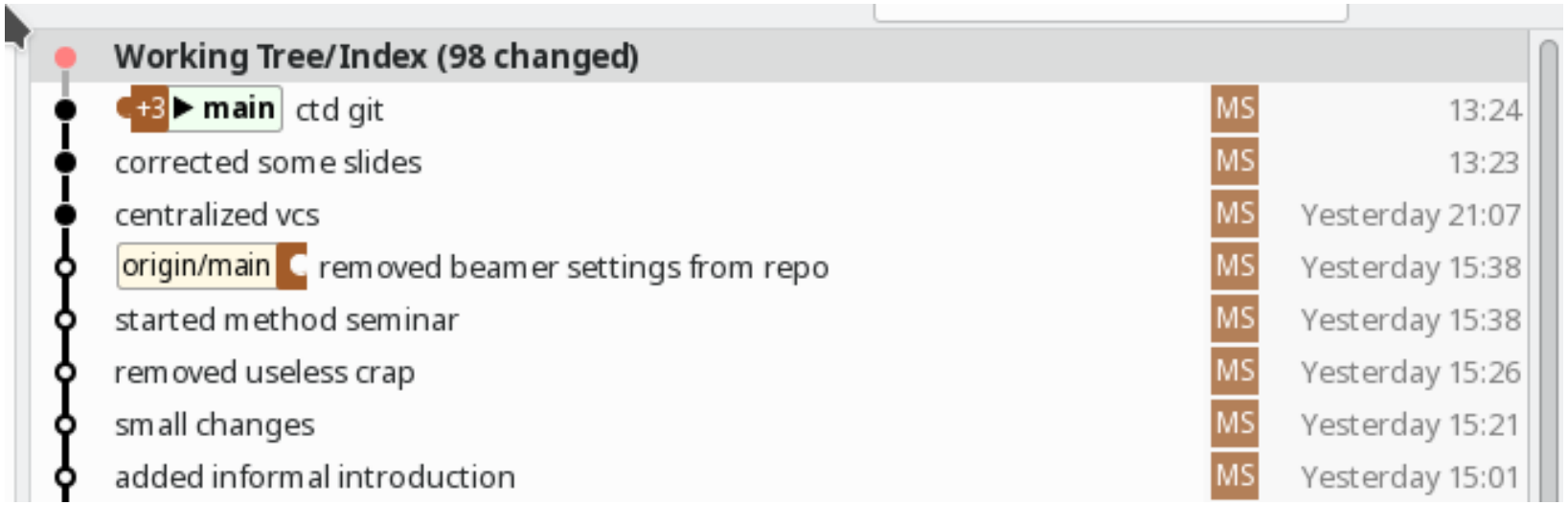
- the command line, typically `git command`, for example `git push`
- a GUI for displaying additional information
  - See `https://git-scm.com/tools/guis` for a list
  - Here : Smartgit (commercial, but free student licences available)



Development history

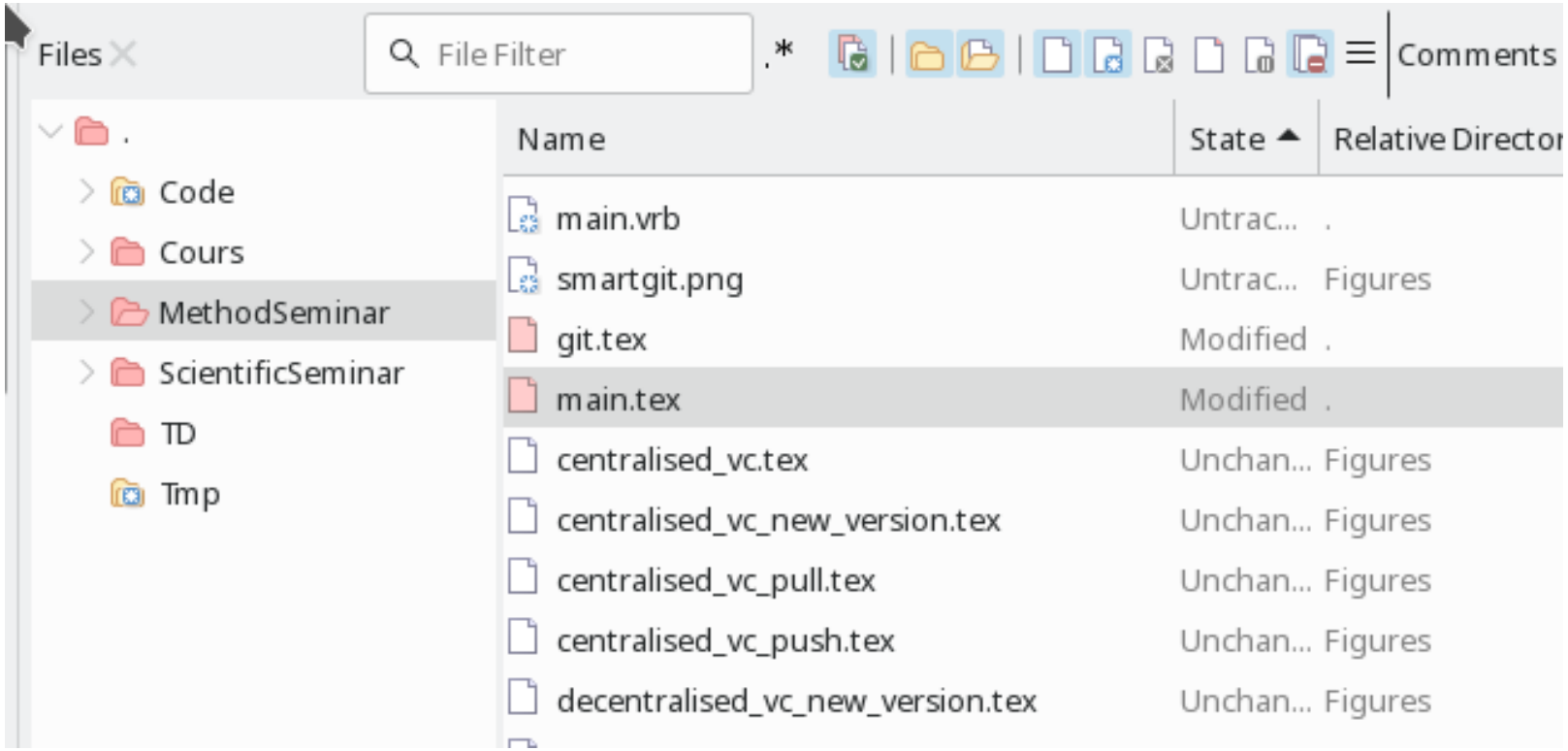
A glimpse at the development history of this lecture series :

- The versions below `origin/main` have already been pushed to the server
- The versions above have been committed but not yet been pushed



Development history

Some files have also been changed locally but not committed yet :



Development history

Through the command line, one obtains almost the same information :

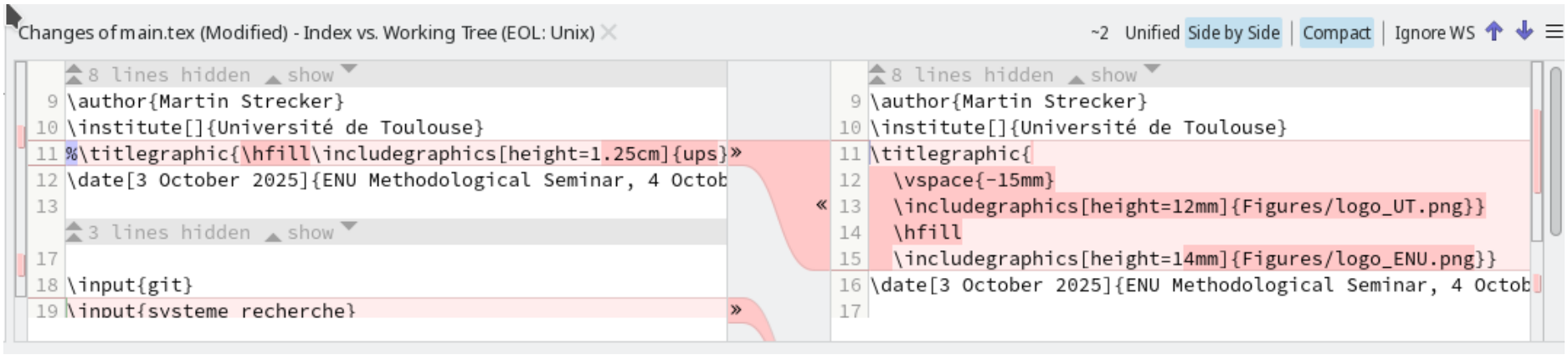
```
strecker@sentosa:~/Teaching/External/Astana_2025/MethodSeminar> git status
On branch main
Your branch is ahead of 'origin/main' by 3 commits.
  (use "git push" to publish your local commits)

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   git.tex
        modified:   main.tex

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        ../Code/
        ../Cours/Figures/Alan_Turing.jpg
        ../Cours/Figures/Alonzo_Church.jpg
        ../Cours/Figures/Haskell_Curry.jpg
```

Development history

Seeing details of changes between commited version and file system version



Technical details

Technical issues :

- After a change, Git only stores the *difference* wrt. the previous version :
  - Efficient for text files
  - Inefficient for binary files / bitmaps : small change = complete replacement
- The local repo is in a `.git` file ; folder can be moved freely in file system

What (not) to put under Git control :

- *Put :* all the files other developers also need :
  code, build files, configuration files
- *Don't put :* anything that is generated :
  - object files / executables in programming projects
  - PDF generated by LaTeX

  because it's useless and creates conflicts

Plan

2 Git
- Git : the version control system
- Actions on a single branch
- Using several branches
- Github

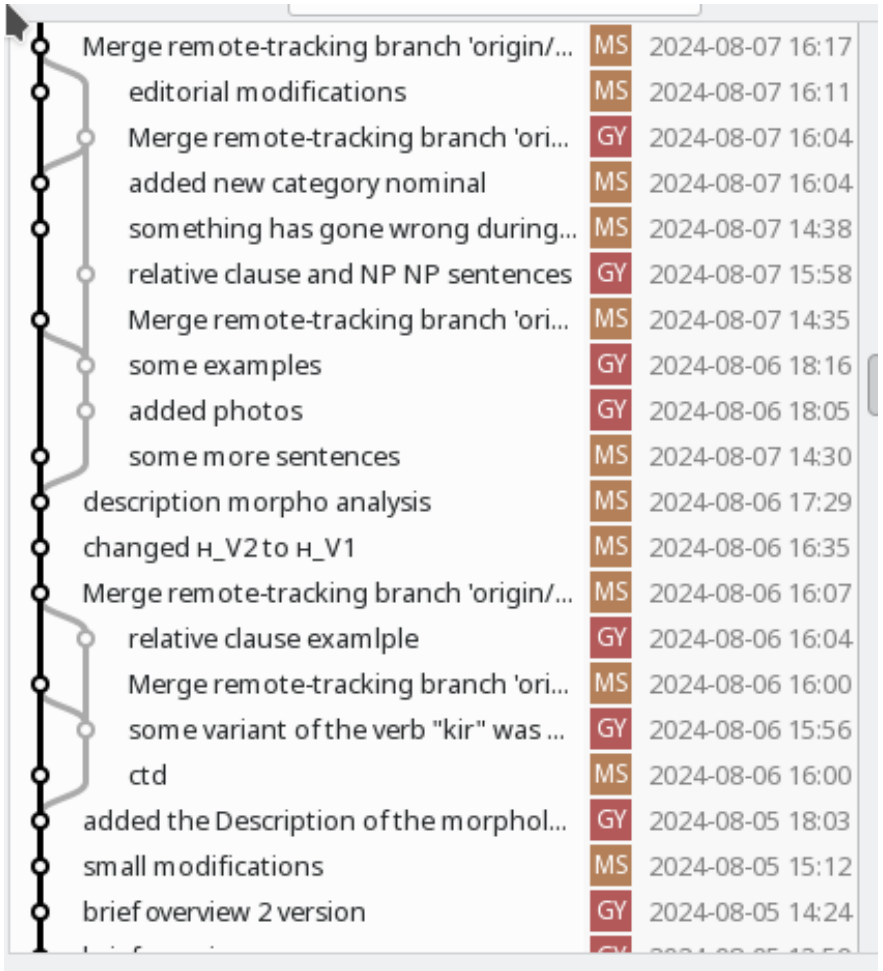
What are branches ?

Typically, several persons collaborate on a project. To work independently without interference by others :

- one can create one or several branches
- then merge them back (typically the `main` branch)

Note :

- Branches are *typical* in collaborative development
- ... but can also be

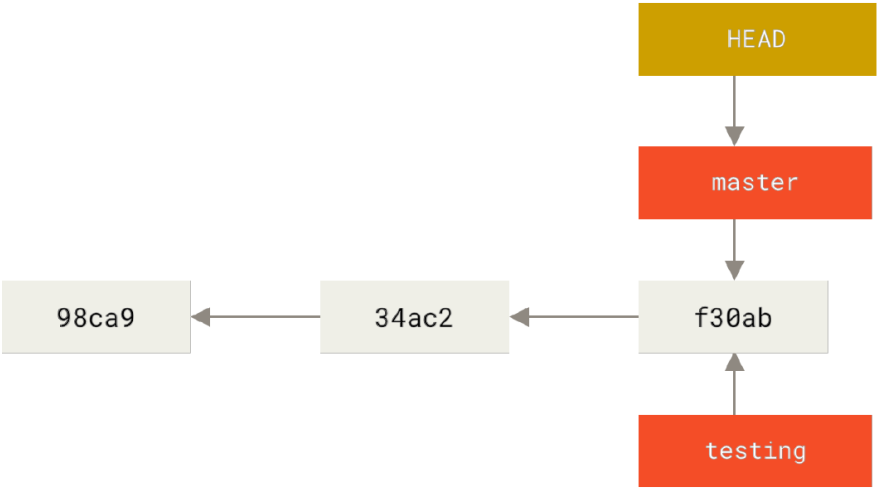


Manipulating branches

(The following diagrams stolen from ProGit book)

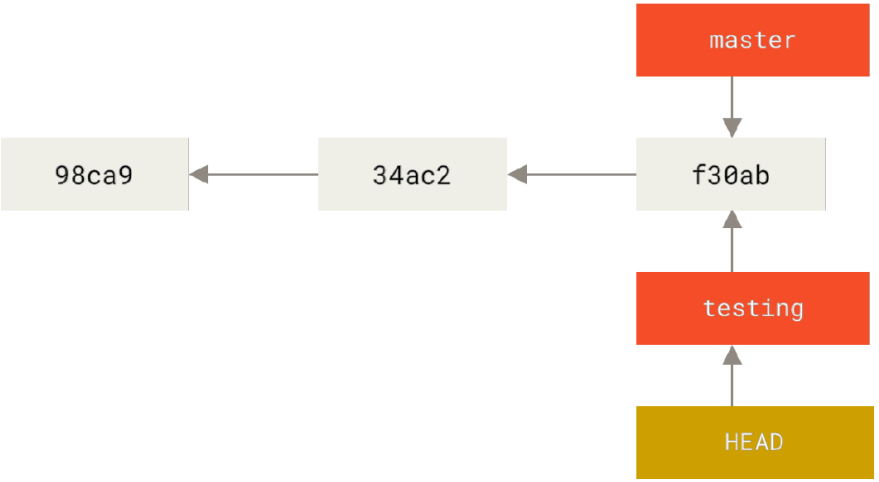
A typical scenario :

- You are on a `master` or `main` branch
- You create a new branch with `git branch testing` (initially with the same contents as `master`)



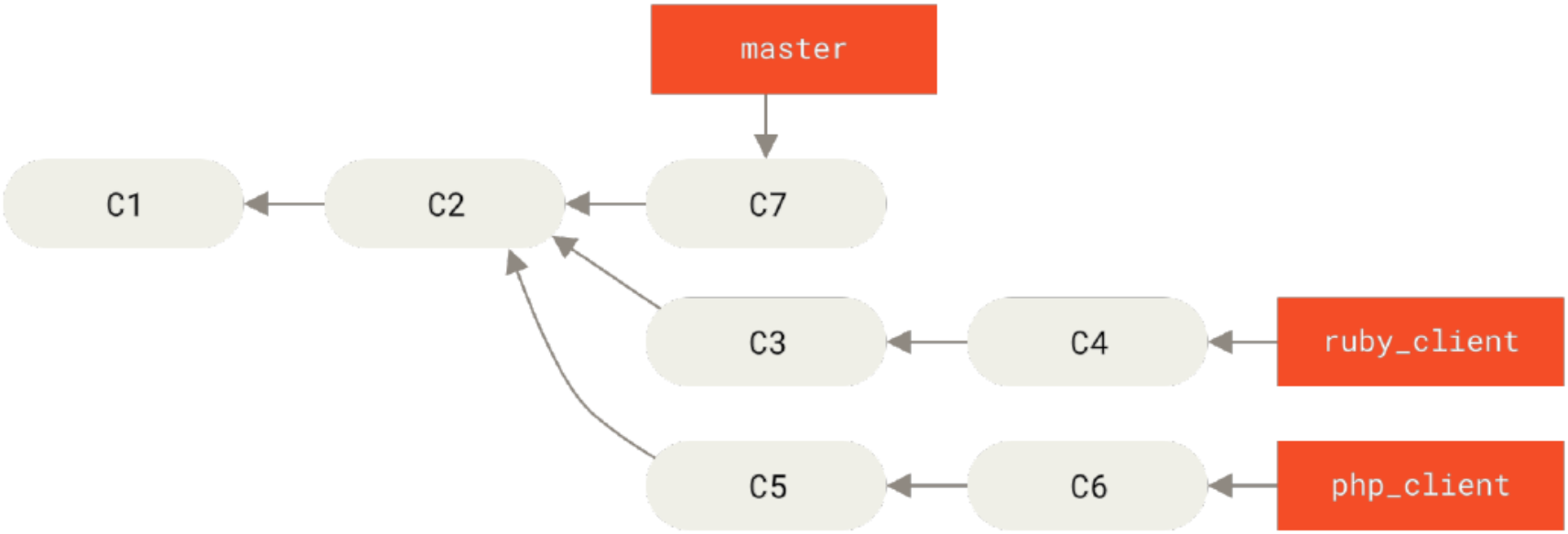
Manipulating branches

- You switch to this branch with `git checkout testing`
- Now, `testing` is the active branch
- . . . whereas `master` is not changed



Manipulating branches

After deleting and creating several branches and pushing versions, you may get :



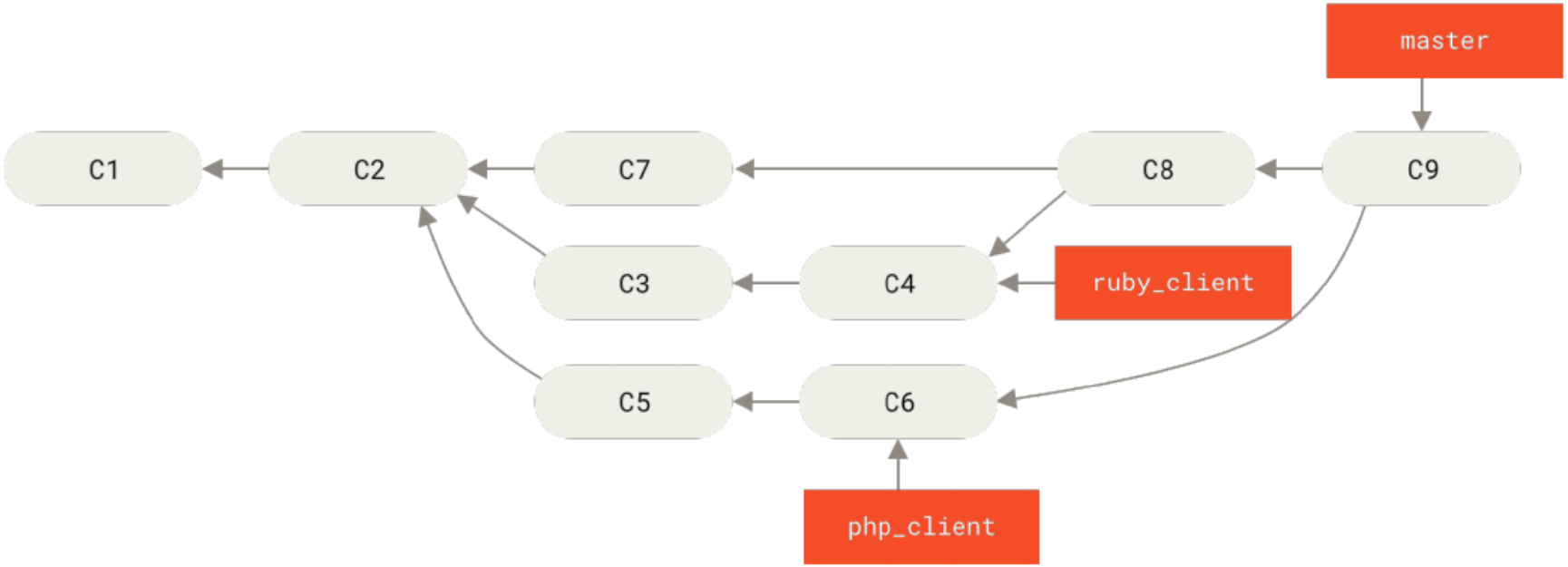
Note :

- One cannot really delete anything from Github
- . . . only make it invisible in the current version
- . . . but it remains in the history

Manipulating branches

The scenario after :

- merging `ruby_client` into `master`
- merging `php_client` into the resulting `master`



Plan

2 Git
- Git : the version control system
- Actions on a single branch
- Using several branches
- Github

Git is not Github

Git is a version control system not requiring a specific provider : You can

- set up your own Git server, or
- use an external hard disk to safely store your repository

Retrieving an existing repository : Cloning with `git clone` *`path`* where *`path`*

- is a local directory, for example
  `git clone /home/Work/Astana_2025`
- an access to a server (via `http` or `ssl`), for example
  `git clone https://github.com/donaldtrump/deals`

Github and Co

Github

- is a hosting platform for Git repositories
- besides other providers for Git : Gitlab etc., see `https://git-scm.com/tools/hosting`
- offering a number of services around code integration, testing, AI support (Copilot)
- . . . but there are other providers on the market, such as Atlassian

Github repositories

Different types of repositories :

- Private (only the owner and invited guests have access)
  Free of charge for up to three collaborators
- Public (everybody can access)

Attention :

- AI tools also sniff around in your private repositories
- . . . so better host critical data on your own servers

Setup

- Create your own Github account
  a bit complex (two-factor authentification)
- Create and clone first repository, see Quickstart